



2025-2 데이터문제해결및실습1

10주차-1

Chapter_07_[경진대회2] 범주형 데이터 이진분류_1

세종대학교

인공지능데이터사이언스학과

박동현 교수



- 본 강의는
골든래빗
출판사의
머신러닝/딥
러닝
문제해결전략
이 제공하는
강의 교안에
기반함.

★★★ 반드시 내 것으로 ★★★

#MUSTHAVE

탄탄한 기본기 + 전략적 사고로 문제해결 역량을 레벨업하자

머신러닝 · 딥러닝 문제해결 전략



Chapter

07

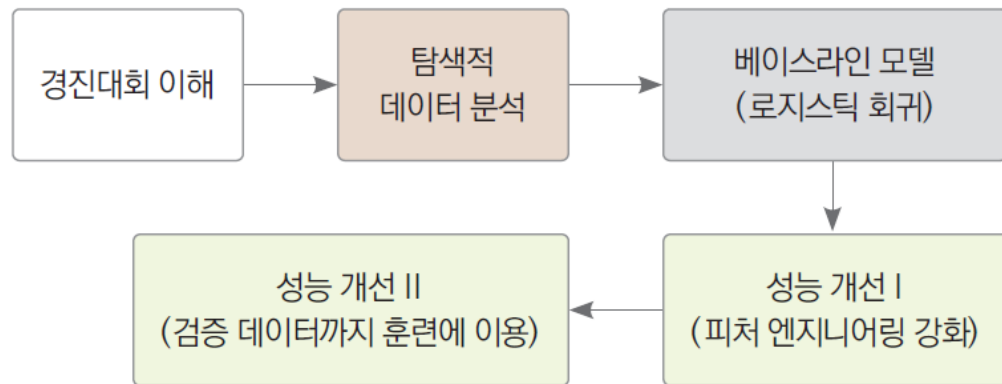
[경진대회]
범주형 데이터 이진분류_1



□ 학습 목표

범주형 데이터를 활용해 이진분류하는 경진대회에 참가해본다. 피처 구성을 이해하기 위해 탐색적 데이터 분석을 자세히 다루며, 모델링 단계에서는 간단한 베이스라인 모델을 먼저 살펴본다. 이어서 베이스라인 모델의 성능을 향상시켜, 최종적으로 프라이빗 리더보드에서 2등을 기록하는 모델을 만들본다. 이번 경진대회에서 데이터 특성에 따른 맞춤형 인코딩 방법을 배울 수 있다.

□ 학습 순서



□ 학습 키워드

- 유형 및 평가지표 : 이진분류, ROC AUC
- 탐색적 데이터 분석 : 피처 요약표, 타깃값 분포, 이진/명목형/순서형/날짜 피처 분포
- 머신러닝 모델 : 로지스틱 회귀
- 피처 엔지니어링 : 원-핫 인코딩, 피처 맞춤 인코딩, 피처 스케일링
- 하이퍼파라미터 최적화 : 그리드서치

□ 핵심 요약

- **피쳐 요약표** : 피쳐별 데이터 타입, 결측값 개수, 고윳값 개수, 실제 입력값 등을 정리한 표
- **타깃값 분포**를 알면 데이터가 얼마나 불균형한지 파악하여 부족한 타깃값에 더 집중해 모델링을 수행할 수 있다.
- **피쳐별로 데이터 특성에 알맞게 인코딩**해줘야 모델 성능을 효과적으로 끌어올릴 수 있다.
 - **이진 피쳐** : 값이 0과 1이 아닌 경우 0과 1로 인코딩한다.
 - **명목형 피쳐** : 고윳값 개수가 너무 많지 않다면 머신러닝 모델이 이해할 수 있도록 원-핫 인코딩을 적용한다.
 - **순서형 피쳐** : 고윳값들의 순서에 맞게 인코딩한다.
 - **날짜 피쳐** : 순환형 데이터는 삼각함수를 사용해 인코딩하거나 원-핫 인코딩을 적용한다.
- **로지스틱 회귀** : 선형 회귀 방식을 응용해 분류 문제에 적용한 모델. 비록 분류 문제에 사용되는 모델이지만 훈련 원리는 선형 회귀 모델과 유사하다.
- **피쳐 스케일링** : 피쳐 간 값의 범위를 일치시키는 작업이다. 피쳐마다 값의 범위가 다르면 훈련이 제대로 안 될 수 있으므로 범위 차이가 심한 피쳐들은 스케일링하여 비슷하게 맞춰줘야 한다.

□ 경진대회 정보

- 난이도 : ★★★
- 경진대회명 : 범주형 데이터 이진분류 경진대회
- 미션 : 다양한 범주형 데이터를 활용해 타깃값 1에 속할 확률 예측
- 문제 유형 : 이진분류
- 평가지표 : ROC AUC
- 데이터 크기 : 64.8MB
- 참가팀 수 : 1,338팀
- 제출 시 사용한 모델 : 로지스틱 회귀
- 파이썬 버전 : 3.7.10
- 사용 라이브러리 및 버전
 - pandas (pandas==1.3.2)
 - seaborn (seaborn==0.11.2)
 - matplotlib (matplotlib==3.4.3)
 - sklearn (scikit-learn==0.23.2)
 - scipy (scipy==1.7.1)

7.1 경진대회 이해

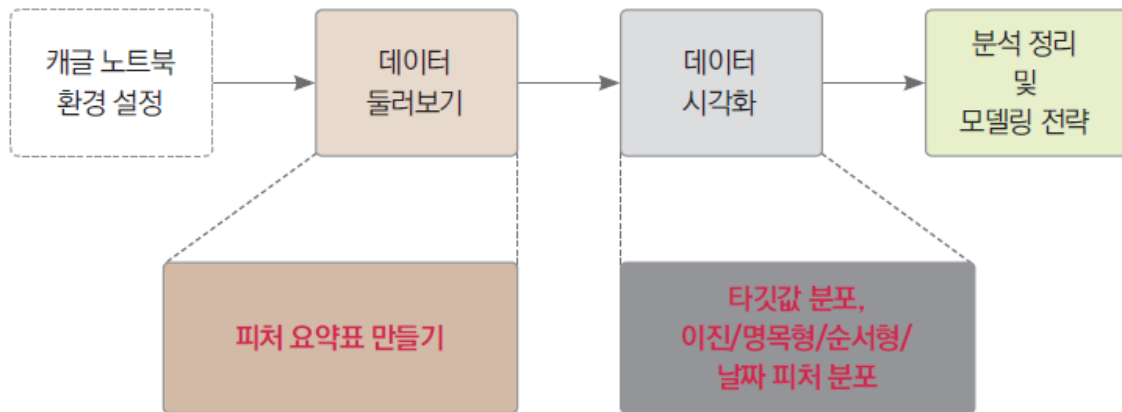
이번 장에서는 범주형 데이터 이진분류 경진대회(Categorical Feature Encoding Challenge)에 참가해본다. 본 경진대회는 2019년 8월부터 12월까지 다섯 달 동안 진행됐으며, 참가한 팀은 총 1,338팀이다. 6장과 마찬가지로 플레이그라운드 대회라 난이도가 쉬운 편이다.

본 대회의 목표는 범주형 피처 23개를 활용해 해당 데이터가 타깃값 1에 속할 확률을 예측하는것이다. 본 경진대회는 특이점이 세 가지 있다.

1. 인위적으로 만든 데이터를 제공한다.
2. 각 피처와 타깃값의 의미를 알 수 없다.
3. 제공되는 데이터는 모두 범주형이다.

7.2 탐색적 데이터 분석

캐글 검색창에 “Categorical Feature Encoding Challenge”를 검색해서 경진대회에 참가한다.



7.2 탐색적 데이터 분석

7.2.1 데이터 둘러보기

- 데이터가 어떻게 구성돼 있는지 살펴보기

```
import pandas as pd
# 데이터 경로
data_path = '/kaggle/input/ds-solution-ch7/'

train = pd.read_csv(data_path + 'train.csv', index_col='id')
test = pd.read_csv(data_path + 'test.csv', index_col='id')
submission = pd.read_csv(data_path + 'sample_submission.csv', index_col='id')
```

- 훈련 데이터 250,000행 24열, 테스트 데이터 50,000행 23열
- target열은 0.5로 일괄 입력됨

7.2 탐색적 데이터 분석

7.2.1 데이터 둘러보기

피처 요약표 만들기

```
def resumetable(df):  
    print(f'데이터셋 형상: {df.shape}')    summary = pd.DataFrame(df.dtypes, columns=['데이터 타입'])  
    summary = summary.reset_index()  
    summary = summary.rename(columns={'index': '피처'})  
    summary['결측값 개수'] = df.isnull().sum().values  
    summary['고유티값 개수'] = df.nunique().values  
    summary['첫 번째 값'] = df.loc[0].values  
    summary['두 번째 값'] = df.loc[1].values  
    summary['세 번째 값'] = df.loc[2].values  
  
    return summary  
  
resumetable(train)
```

7.2 탐색적 데이터 분석

7.2.1 데이터 둘러보기

피처 요약표 해석하기

- 이진 피처 요약표 (bin_0~bin_4)

	피처	데이터 타입	결측값 개수	고유티 값 개수	첫 번째 값	두 번째 값	세 번째 값
0	bin_0	int64	0	2	0	0	0
1	bin_1	int64	0	2	0	1	0
2	bin_2	int64	0	2	0	0	0
3	bin_3	object	0	2	T	T	F
4	bin_4	object	0	2	Y	Y	Y

7.2 탐색적 데이터 분석

7.2.1 데이터 둘러보기

피처 요약표 해석하기

- 명목형 피처 요약표 (nom_0~nom_9)

5	nom_0	object	0	3	Green	Green	Blue
6	nom_1	object	0	6	Triangle	Trapezoid	Trapezoid
7	nom_2	object	0	6	Snake	Hamster	Lion
8	nom_3	object	0	6	Finland	Russia	Russia
9	nom_4	object	0	4	Bassoon	Piano	Theremin
10	nom_5	object	0	222	50f116bcf	b3b4d25d0	3263bdce5
11	nom_6	object	0	522	3ac1b8814	fbcb50fc1	0922e3cb8
12	nom_7	object	0	1220	68f6ad3e9	3b6dd5612	a6a36f527
13	nom_8	object	0	2215	c389000ab	4cd920251	de9c9f684
14	nom_9	object	0	11981	2f4cb3d51	f83c56c21	ae680dd0

7.2 탐색적 데이터 분석

7.2.1 데이터 둘러보기

피처 요약표 해석하기

- 순서형 피처 요약표 (ord_0~ord_5)

15	ord_0	int64	0	3	2	1	1
16	ord_1	object	0	5	Grandmaster	Grandmaster	Expert
17	ord_2	object	0	6	Cold	Hot	Lava Hot
18	ord_3	object	0	15	h	a	h
19	ord_4	object	0	26	D	A	R
20	ord_5	object	0	192	kr	bF	Jc

- 그 외 피처(day, month, target)

21	day	int64	0	7	2	7	7
22	month	int64	0	12	2	8	2
23	target	int64	0	2	0	0	0

7.2 탐색적 데이터 분석

7.2.2 데이터 시각화

- 피쳐들을 시각화하여 타깃값별 피쳐 분포를 알아본다. 어떤 피쳐가 중요하고 어떤 고윳값이 타깃값에 영향을 많이 주는지 알 수 있다.
- 시각화 라이브러리 불러오기

```
import seaborn as sns
import matplotlib as mpl
import matplotlib.pyplot as plt
%matplotlib inline
```

7.2 탐색적 데이터 분석

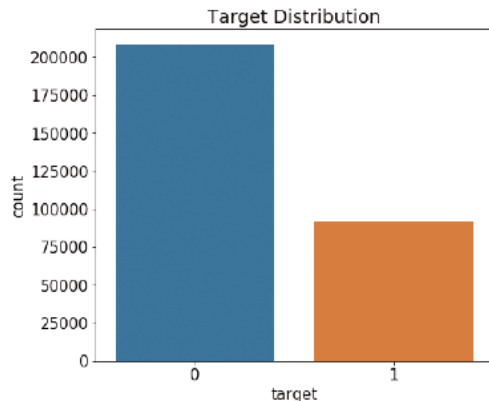
7.2.2 데이터 시각화

타겟값 분포

타겟값 분포를 알면 데이터가 얼마나 불균형한지 파악하여 부족한 타겟값에 더 집중해 모델링을 수행할 수 있다.

```
mpl.rcParams['font', size=15) # 폰트 크기 설정
plt.figure(figsize=(7, 6)) # Figure 크기 설정

# 타겟값 분포 카운트플롯
ax = sns.countplot(x='target', data=train)
ax.set_title('Target Distribution');
```

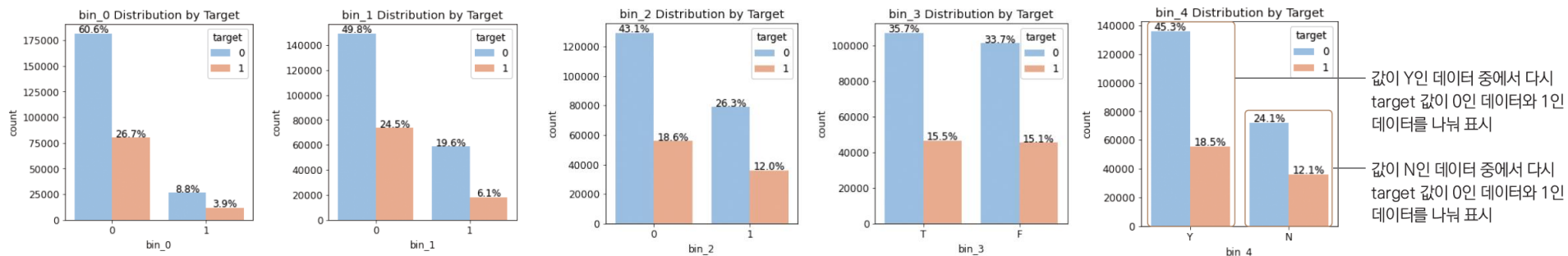


7.2 탐색적 데이터 분석

7.2.2 데이터 시각화

이진 피처 분포

: 이진 피처의 분포를 타깃값별로 따로 그린다. 특정 고윳값이 특정 타깃값에 치우치는지 확인할 수 있다.



값이 Y인 데이터 중에서 다시 target 값이 0인 데이터와 1인 데이터를 나눠 표시

값이 N인 데이터 중에서 다시 target 값이 0인 데이터와 1인 데이터를 나눠 표시

7.2 탐색적 데이터 분석

7.2.2 데이터 시각화

여러 그래프를 격자 형태로 배치하는 GridSpec 사용한다.

```
import matplotlib.gridspec as gridspec # 여러 그래프를 격자 형태로 배치
# 3행 2열 틀(Figure) 준비
mpl.rcParams['font', size=12]
grid = gridspec.GridSpec(3, 2) # 그래프(서브플롯)를 3행 2열로 배치 1
plt.figure(figsize=(10, 16)) # 전체 Figure 크기 설정
plt.subplots_adjust(wspace=0.4, hspace=0.3) # 서브플롯 간 좌우/상하 여백 설정 2

# 서브플롯 그리기
bin_features = ['bin_0', 'bin_1', 'bin_2', 'bin_3', 'bin_4'] # 피쳐 목록 3

for idx, feature in enumerate(bin_features): # 4
    ax = plt.subplot(grid[idx]) # 5

    # ax축에 타깃값 분포 카운트플롯 그리기 6
    sns.countplot(x=feature,
                  data=train,
                  hue='target',
                  palette='pastel', # 그래프 색상 설정
                  ax=ax)

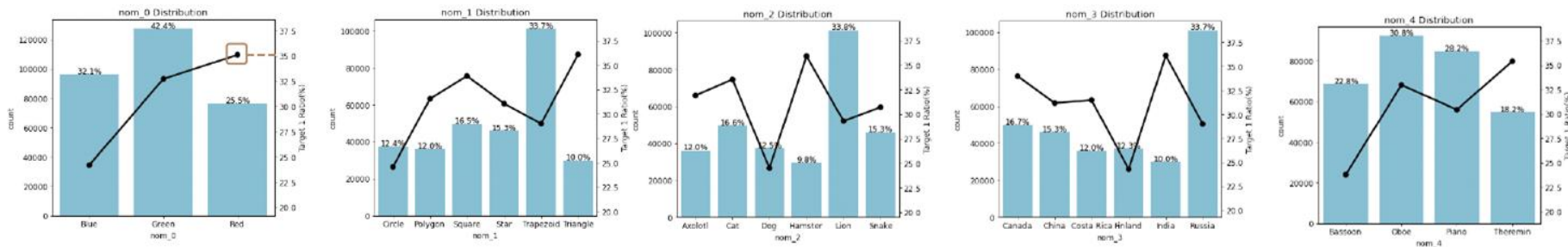
    ax.set_title(f'{feature} Distribution by Target') # 그래프 제목 설정 7
    write_percent(ax, len(train)) # 비율 표시 8
```

7.2 탐색적 데이터 분석

7.2.2 데이터 시각화

명목형 피쳐 분포

: 명목형 피쳐 분포와 명목형 피쳐별 타깃값 1의 비율을 살펴본다. 각 그림에서 카운트플롯은 피쳐별 고깃값의 비율을 나타낸다. 꺾은 선 그래프는 포인트 플롯으로, 해당 고깃값 중 타깃값이 1의 비율을 나타낸다.



7.2 탐색적 데이터 분석

7.2.2 데이터 시각화

스텝 1: 교차분석표 생성 함수 만들기

- 교차표(cross-tabulation) 혹은 교차분석표 : 범주형 데이터 2개를 비교 분석하는 데 사용되는 표로, 각 범주형 데이터의 빈도나 통계량을 행과 열로 결합해놓은 표

- 판다스의 `crosstab()` 함수로 교차분석표 생성

```
pd.crosstab(train['nom_0'], train['target'])
```

- 백분율로 표현하기 위해 정규화된 교차분석표에 100을 곱하기

```
# 정규화 후 비율을 백분율로 표현
crosstab = pd.crosstab(train['nom_0'], train['target'], normalize='index') * 100
crosstab
```

- 인덱스 재설정

```
crosstab = crosstab.reset_index() # 인덱스 재설정
crosstab
```

7.2 탐색적 데이터 분석

7.2.2 데이터 시각화

스텝 2: 포인트플롯 생성 함수 만들기

- 스텝 1에서 구한 교차분석표를 사용하여 타깃값 1의 비율을 나타내는 포인트플롯을 그리는 함수를 만든다. 함수 이름은 `plot_pointplot()`이며 다음의 세 파라미터를 받는다.
 - `ax`: 포인트플롯을 그릴 축
 - `feature`: 포인트플롯으로 그릴 피처
 - `crosstab`: 교차분석표
- `plot_pointplot()`은 이미 카운트플롯이 그려진 축에 포인트플롯을 중복으로 그린다.

```
def plot_pointplot(ax, feature, crosstab):
    ax2 = ax.twinx() # x축은 공유하고 y축은 공유하지 않는 새로운 축 생성
    # 새로운 축에 포인트플롯 그리기
    ax2 = sns.pointplot(x=feature, y=1, data=crosstab,
                        order=crosstab[feature].values, # 포인트플롯 순서
                        color='black', # 포인트플롯 색상
                        legend=False) # 범례 미표시
    ax2.set_ylim(crosstab[1].min()-5, crosstab[1].max()*1.1) # y축 범위 설정
    ax2.set_ylabel('Target 1 Ratio(%)')
```

7.2 탐색적 데이터 분석

7.2.2 데이터 시각화

스텝 3: 피쳐 분포도 및 피쳐별 타깃값 1의 비율 포인트플롯 생성 함수 만들기

- `get_crosstab()`과 `plot_pointplot()` 함수를 활용해 최종적인 그래프를 그리는 함수를 만든다.

```
def plot_cat_dist_with_true_ratio(df, features, num_rows, num_cols,
                                  size=(15, 20)):
    plt.figure(figsize=size) # 전체 Figure 크기 설정
    grid = gridspec.GridSpec(num_rows, num_cols) # 서브플롯 배치
    plt.subplots_adjust(wspace=0.45, hspace=0.3) # 서브플롯 좌우/상하 여백 설정
    for idx, feature in enumerate(features): #
        ax = plt.subplot(grid[idx])
        crosstab = get_crosstab(df, feature) # 교차분석표 생성

        # ax축에 타깃값 분포 카운트플롯 그리기
        sns.countplot(x=feature, data=df,
                      order=crosstab[feature].values,
                      color='skyblue',
                      ax=ax)

        write_percent(ax, len(df)) # 비율 표시
        plot_pointplot(ax, feature, crosstab) # 포인트플롯 그리기
        ax.set_title(f'{feature} Distribution') # 그래프 제목 설정
```

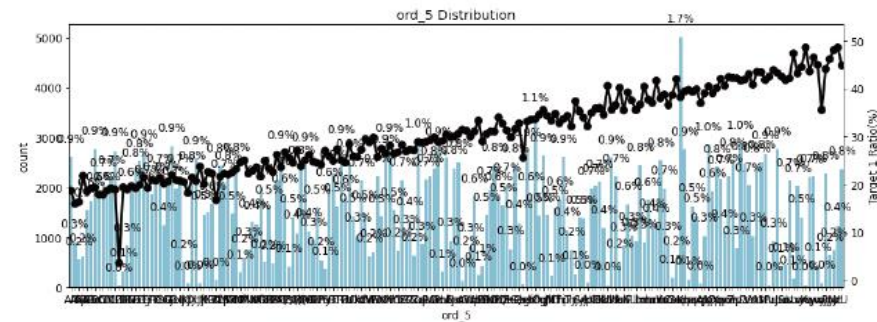
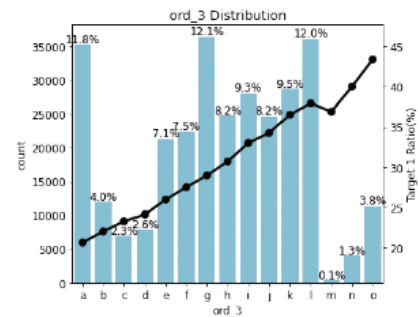
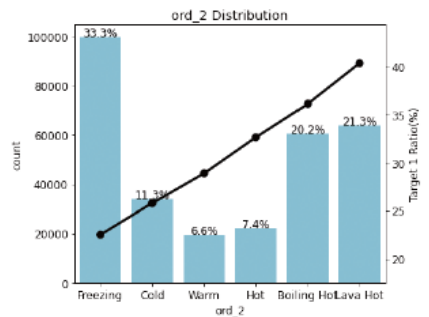
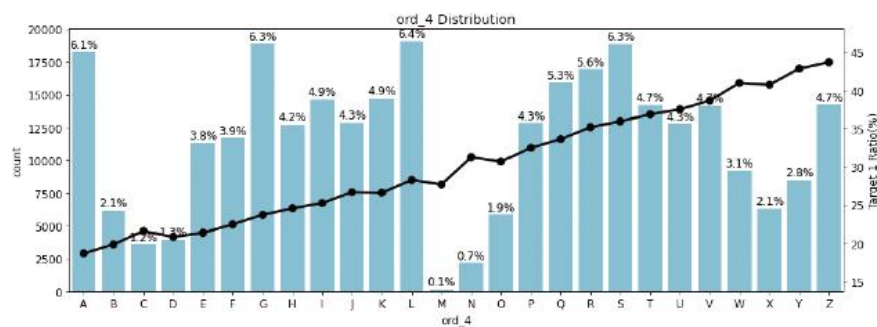
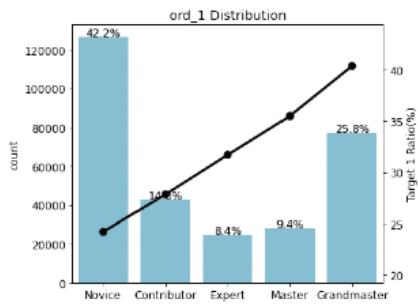
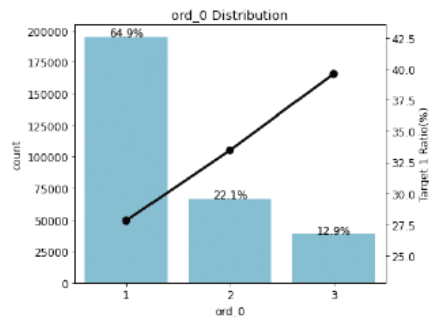
7.2 탐색적 데이터 분석

7.2.2 데이터 시각화

순서형 피쳐 분포

- : `plot_cat_dist_with_true_ratio()` 함수를 사용해서 순서형 피쳐 분포도 살펴본다. 순서형 피쳐는 총 6개다.
- 7.2.1절의 ‘순서형 피쳐 요약표’에서 확인했듯이 `ord_0`부터 `ord_3`까지는 고윳값 개수가 15개 이하인 반면 `ord_4`와 `ord_5`는 고윳값이 훨씬 많다. `ord_0`부터 `ord_3`까지는 2행 2열로 그래프를 그리고, `ord_4`와 `ord_5`는 2행 1열로 그래프를 따로 그린다.
- 파라미터 설정
 - `categories` : 범주형 데이터 타입으로 인코딩할 값 목록
 - `ordered` : `True`로 설정하면 `categories`에 전달한 값의 순서가 유지된다.

7.2 탐색적 데이터 분석

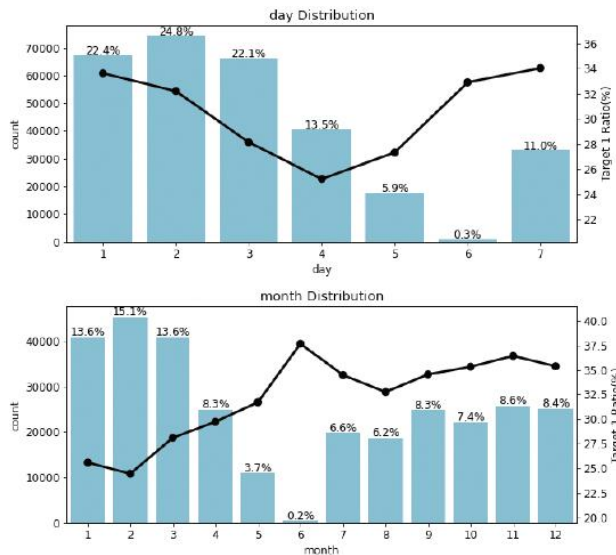


7.2 탐색적 데이터 분석

7.2.2 데이터 시각화

날짜 피쳐 분포

```
date_features = ['day', 'month']  
plot_cat_dist_with_true_ratio(train, date_features,  
num_rows=2, num_cols=1, size=(10, 10))
```



분석 정리 및 모델링 전략

분석 정리

1. 결측값은 없다.
2. 모든 피처가 중요하여 제거할 피처를 찾지 못했다.
3. 이진 피처 인코딩 : 값이 숫자가 아닌 이진 피처는 0과 1로 인코딩한다.
4. 명목형 피처 인코딩 : 전체 데이터가 크지 않으므로 모두 원-핫 인코딩한다.
5. 순서형 피처 인코딩 : 고유타값들의 순서에 맞게 인코딩한다(이미 숫자로 되어 있다면 인코딩필요 없음).
6. 날짜 피처 인코딩 : 값의 크고 작음으로 해석되지 못하도록 원-핫 인코딩한다.

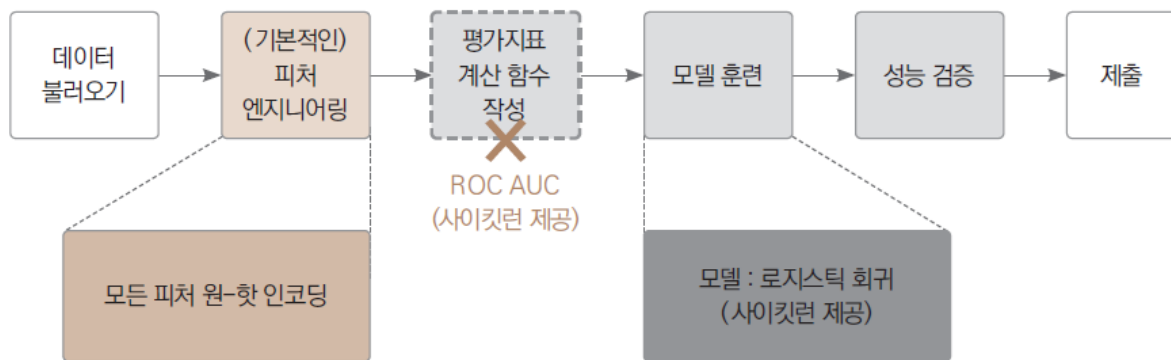
분석 정리 및 모델링 전략

모델링 전략

- 베이스라인 모델 : 로지스틱 회귀 모델
 - 피처 엔지니어링 : 모든 피처를 원-핫 인코딩
- 성능 개선 : 추가 피처 엔지니어링과 하이퍼파라미터 최적화
 - 피처 엔지니어링 : 피처 맞춤 인코딩과 피처 스케일링
 - 하이퍼파라미터 최적화 : 그리드서치
 - 추가 팁 : 검증 데이터를 훈련에 이용

7.3 베이스라인 모델

모든 피처를 원-핫 인코딩한 뒤, 로지스틱 회귀 모델13로 베이스라인을 만들어본다. 전체 프로세스는 6장과 같다.



7.3 베이스라인 모델

7.3.1 피처 엔지니어링

데이터 합치기

- 머신러닝 모델은 문자 데이터를 인식하지 못하므로 문자를 숫자로 바꿔야 한다. 이처럼 데이터의 표현 형태를 바꾸는 작업을 **인코딩**이라고 한다.
- 훈련 데이터와 테스트 데이터에 동일한 인코딩을 적용하기 위해 편의상 둘을 합쳐 진행한다. 그런 다음 합친 DataFrame에서 타깃값을 제거한다. 피처와 타깃값은 따로 분리해서 모델링해야 하기 때문이다.

```
all_data = pd.concat([train, test]) # 훈련 데이터와 테스트 데이터 합치기
all_data = all_data.drop('target', axis=1) # 타깃값 제거
all_data
```

7.3 베이스라인 모델

7.3.1 피처 엔지니어링

원-핫 인코딩

- 모델링에 사용하기 위해 모든 피처를 원-핫 인코딩한다. 준비한 all_data에 원-핫 인코딩을 적용한다.

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder() # 원-핫 인코더 생성 1
all_data_encoded = encoder.fit_transform(all_data) # 원-핫 인코딩 적용 2
```

7.3 베이스라인 모델

7.3.1 피처 엔지니어링

데이터 나누기

- 공통으로 적용할 인코딩이 끝났으니 훈련 데이터와 테스트 데이터를 다시 나눈다. 행 번호를 기준으로 다시 나눌 수 있다.

```
num_train = len(train) # 훈련 데이터 개수

# 훈련 데이터와 테스트 데이터 나누기
X_train = all_data_encoded[:num_train] # 0~num_train - 1행
X_test = all_data_encoded[num_train:] # num_train~마지막 행

y = train['target']
```

7.3 베이스라인 모델

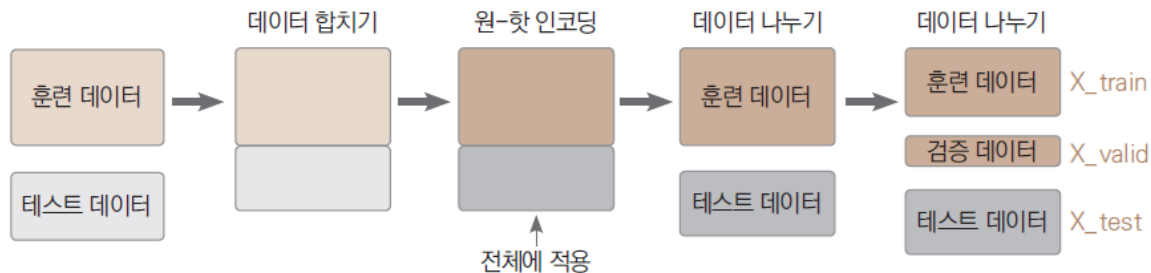
7.3.1 피처 엔지니어링

데이터 나누기

- 훈련 데이터에서 일부를 검증 데이터로 나눈다. 훈련 데이터는 모델 훈련에 사용하고, 검증 데이터는 모델 성능 검증에 사용한다.

```
from sklearn.model_selection import train_test_split

# 훈련 데이터, 검증 데이터 분리
X_train, X_valid, y_train, y_valid = train_test_split(X_train, y,
                                                    test_size=0.1,
                                                    stratify=y,
                                                    random_state=10)
```



7.3 베이스라인 모델

7.3.2 모델 훈련

- 이제 모델을 생성한 뒤, 앞에서 준비한 데이터를 사용해서 훈련한다. 선형 회귀 방식을 응용해 분류를 수행하는 로지스틱 회귀 모델을 사용한다.

```
from sklearn.linear_model import LogisticRegression

logistic_model = LogisticRegression(max_iter=1000, random_state=42) # 모델 생성
logistic_model.fit(X_train, y_train) # 모델 훈련
```

- 베이스라인 모델의 파라미터는 간단하게 설정한다.
 - `max_iter` : 모델의 회귀 계수를 업데이트하는 반복 횟수. 모델 훈련 시 회귀 계수를 업데이트하면서 훈련하는 데, 이때 업데이트를 몇 번 할지를 정한다.
 - `random_state` : 값을 지정하면 여러 번 실행해도 매번 똑같은 결과가 나온다. 아무 값으로 지정해도 상관없다.

7.3 베이스라인 모델

7.3.3 모델 성능 검증

- 사이킷런은 타깃값 예측 메서드를 두 가지 제공한다. `predict()`와 `predict_proba()`다. 먼저 `predict()` 메서드는 타깃값 자체(0이나 1이나)를 예측한다. 반면, `predict_proba()`는 타깃값의 확률(0일 확률과 1일 확률)을 예측한다.

- `predict_proba()`로 검증 데이터의 타깃값이 0 또는 1일 확률을 예측한다.

```
logistic_model.predict_proba(X_valid)
```

- `predict()` 메서드로 타깃값을 예측한다.

```
logistic_model.predict(X_valid)
```

- 검증 데이터를 활용해 타깃값을 예측한다.

```
# 검증 데이터를 활용한 타깃 예측  
y_valid_preds = logistic_model.predict_proba(X_valid)[:, 1]
```

7.3 베이스라인 모델

7.3.3 모델 성능 검증

- 타깃 예측값인 `y_valid_preds`와 실제 타깃값인 `y_valid`를 이용해 ROC AUC를 구한다. 모델 성능을 검증하려는 절차다. ROC AUC 점수는 사이킷런의 `roc_auc_score()` 함수를 이용하면 쉽게 구할 수 있다.

```
from sklearn.metrics import roc_auc_score # ROC AUC 점수 계산 함수

# 검증 데이터 ROC AUC
roc_auc = roc_auc_score(y_valid, y_valid_preds)

print(f'검증 데이터 ROC AUC : {roc_auc:.4f}')
```

7.3 베이스라인 모델

7.3.4 예측 및 결과 제출

- ‘테스트 데이터’를 활용해 타깃값이 1일 확률을 예측하고, 결과를 제출한다.

```
# 타깃값 1일 확률 예측
y_preds = logistic_model.predict_proba(X_test)[:, 1]
```

- 제출 파일을 만든다.

```
# 제출 파일 생성
submission['target'] = y_preds
submission.to_csv('submission.csv')
```

- 커밋 후 제출한다.